DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA09 - Programming in Java - Slot A

FINAL ASSIGNMENT REPORT

Real-Time Railway Ticket Reservation System:

Concurrent Multi-Threaded Reservation, Synchronization, AWT GUI & JDBC Persistence

Course Code & Name	**CSA09 - Programming in Java**
Slot	**Slot A**
Course Outcome (CO)	**CO4 - Develop robust multi-threaded and database-driven enterprise solutions by implementing object-oriented operations, collections, synchronization, and GUI architectures in Java for real-world applications.**
Bloom's Taxonomy Level	**Level 4 (L4) - Analyze**
SDG Mapping	**SDG 8 (Decent Work & Economic Growth), SDG 9 (Industry, Innovation & Infrastructure), SDG 11 (Sustainable Cities & Communities)**
Student Name	**MRINALLI S**
Register Number	**1924111S75**
Programming Language	**Java (JDK 21+ / SE Standard)**
Submission Date	**September 2026**

1. Problem Statement

Modern railway transportation systems serve millions of passengers through railway stations, online reservation portals, mobile applications, and computerized booking centers. In such high-demand environments, railway reservation systems face significant technical challenges related to real-time seat availability, concurrent booking requests, data consistency, synchronization, transaction reliability, and persistent storage of passenger and ticket information.

A fundamental challenge in a multi-user railway reservation system is the possibility of race conditions and duplicate seat allocation. When multiple passengers attempt to book the same seat simultaneously without proper synchronization, the system may incorrectly assign the same seat to more than one passenger. For example, if two passengers simultaneously attempt to reserve Seat A1 on the same train, both booking requests may read the seat as available before either request updates its status. Without coordinated access and thread synchronization, both passengers could receive successful booking confirmations, resulting in duplicate ticket generation and inconsistent reservation records.

Furthermore, a modern railway reservation architecture requires a clear separation of responsibilities across different railway domains, including passenger management, train information, coach and seat management, ticket booking and cancellation, payment processing, exception handling, graphical user interfaces, concurrent booking operations, file-based backup, and relational database storage. The system must maintain accurate reservation information while supporting multiple users and ensuring that each seat is allocated to only one passenger at a time.

The proposed Real-Time Railway Ticket Reservation System must therefore provide a comprehensive Java-based solution that integrates Object-Oriented Programming principles, Java Collections, multithreading, synchronization mechanisms, graphical user interface programming, file handling, object serialization, and JDBC database connectivity.

The problem requires us to:

- Design a comprehensive and modular Object-Oriented railway domain model in Java that encapsulates passengers, trains, coaches, seats, tickets, payments, and reservation records.
- Implement suitable class hierarchies using inheritance, abstract classes, interfaces, encapsulation, method overloading, method overriding, and runtime polymorphism to represent real-world railway reservation entities and operations.
- Utilize Java Collections such as HashMap for efficient train and ticket lookup, ArrayList for storing ordered passenger and booking information, HashSet for preventing duplicate records, along with type-safe Generics and Iterators for efficient data processing.
- Formulate robust custom exception hierarchies to detect and gracefully handle invalid passenger information, unavailable seats, invalid booking requests, duplicate bookings, and invalid ticket cancellation operations.
- Engineer a concurrent booking system using Java Threads and Runnable tasks to simulate multiple passengers attempting to reserve seats simultaneously, while employing synchronized critical sections to prevent race conditions and duplicate seat allocation.
- Implement inter-thread communication using wait() and notifyAll() mechanisms to coordinate booking-related operations and maintain proper synchronization between concurrent processes.

- Construct a native Java AWT graphical user interface incorporating Frame, Panel, Label, TextField, TextArea, Button, ScrollPane, layout managers, and ActionListener-based event handling for passenger reservation operations.
- Implement a reliable file persistence mechanism using Java Object Serialization to save and restore reservation data through ObjectOutputStream and ObjectInputStream.
- Develop a relational database persistence layer using JDBC and PreparedStatement operations to perform secure CRUD operations, including INSERT, SELECT, UPDATE, and DELETE, for passengers, trains, seats, tickets, and payment information.
- Evaluate the performance and reliability of the system by analyzing time and space complexity, synchronization mechanisms, data consistency, booking accuracy, database persistence, and overall scalability of the railway reservation architecture.

---

2. Objective

- **Primary Objective:** Architect, implement, and rigorously validate a robust, real-time Railway Ticket Reservation System in Java that integrates Object-Oriented Programming, concurrent booking management, thread synchronization, Java AWT graphical interfaces, file serialization, and JDBC database persistence.
- **Java Programming Mastery:** Demonstrate comprehensive knowledge of Java programming concepts, including Object-Oriented Programming, encapsulation, inheritance, polymorphism, abstract classes, interface contracts, method overloading, method overriding, custom exception handling, and the Java Collections Framework.
- **Real-Time Reservation Management:** Develop a reliable reservation system that enables passengers to search for trains, check seat availability, book tickets, view reservation details, cancel tickets, and maintain accurate real-time booking records.
- **Concurrent Booking Engine:** Engineer a thread-safe booking mechanism using Java Threads, Runnable tasks, synchronized critical sections, and thread coordination to ensure that multiple passengers cannot successfully reserve the same seat simultaneously.
- **Race Condition Prevention:** Implement synchronization mechanisms that guarantee atomic seat booking operations and maintain consistent seat availability information even when multiple booking requests occur at the same time.
- **Inter-Thread Communication:** Implement Java monitor primitives such as wait() and notifyAll() to demonstrate controlled communication and coordination between concurrent booking processes.
- **Graphical User Interface:** Design a functional and user-friendly native Java AWT desktop application using Frame, Panel, GridLayout, FlowLayout, BorderLayout, TextField, TextArea, Button, ScrollPane, and ActionListener-based event processing.
- **Java Collections Integration:** Efficiently manage railway reservation data using ArrayList, HashMap, HashSet, Generics, and Iterators for storing, searching, organizing, and processing passenger, train, seat, and ticket information.
- **Sustainable Development Contribution:** Support relevant United Nations Sustainable Development Goals, particularly **SDG 8 (Decent Work and Economic Growth)** by improving the efficiency of transportation-related services, **SDG 9 (Industry, Innovation and Infrastructure)** through reliable and scalable digital railway infrastructure, and **SDG 11 (Sustainable Cities and Communities)** by supporting efficient and accessible transportation management through computerized railway reservation services.

3. Requirements and Environment Used

3.1 Software & Hardware Environment

Component	Specification / Tool
Operating System	Windows 11 64-bit / Ubuntu 22.04 LTS / macOS
Programming Language	Java SE (Standard Edition) – JDK 21+
Primary Compiler	javac / OpenJDK 64-Bit Server VM
Runtime Environment	Java Virtual Machine (JVM) with HotSpot JIT Engine
Database Engine	MySQL 8.0 Relational Database Management System
JDBC Driver	mysql-connector-j-8.x.jar
GUI Toolkit	Java Abstract Window Toolkit (java.awt*., java.awt.event.*)
Development IDE	Visual Studio Code / IntelliJ IDEA / Eclipse IDE for Java Developers
Build & Execution	javac and java commands
Version Control	Git with GitHub repository tracking
Memory and Performance Tools	JVM VisualVM / JConsole / Java Flight Recorder (JFR)
Hardware Requirements	Minimum 4 GB RAM, Dual-Core Processor, 2 GB Free Disk Space

Example Build and Execution Command:

```
javac -cp ".;mysql-connector-j-8.x.jar" src/main/Main.java

java -cp ".;mysql-connector-j-8.x.jar" main.Main
```

3.2 Java Concepts, Data Structures & Standard Libraries Used

Construct / Library	Architectural Role & System Application
Classes & Objects	Encapsulates real-world railway entities including Passenger, Train, Coach, Seat, Ticket, Payment, and Reservation System.
Abstract Class & Interface	An abstract User class defines common user properties, while the RailwayOperations interface specifies core operations such as train search, seat availability checking, ticket booking, cancellation, and booking retrieval.

Construct / Library	Architectural Role & System Application
Inheritance & Polymorphism	Passenger classes inherit common properties from the User base class. Runtime polymorphism enables railway operations to be handled through common interface references.
HashMap	Provides O(1) average-time lookup for trains using train numbers and tickets using unique ticket IDs.
ArrayList	Dynamically stores ordered collections of passengers, coaches, seats, and booking records.
HashSet	Prevents duplicate passenger IDs and duplicate booking or ticket identifiers.
Java Generics & Iterator	Generics provide compile-time type safety, while Iterator enables safe traversal of booking and passenger records.
Custom Exception Hierarchy	InvalidPassengerException, SeatUnavailableException, InvalidBookingException, DuplicateBookingException, and InvalidCancellationException handle domain-related failures gracefully.
Multithreading & Runnable	Concurrent booking requests are simulated using BookingTask worker threads representing multiple passengers attempting reservations simultaneously.
Synchronization & Locking	synchronized methods or monitor blocks enforce mutual exclusion during seat allocation, ensuring that only one passenger can reserve a particular seat.
wait() & notifyAll()	Inter-thread communication coordinates waiting booking operations and notifies waiting threads when reservation or seat status changes.
Java AWT GUI & Listeners	A native desktop reservation interface is developed using Frame, Panel, Label, TextField, TextArea, Button, ScrollPane, layout managers, and ActionListener event handling.
Object Serialization	ObjectOutputStream and ObjectInputStream save and restore complete railway reservation data from persistent files.
JDBC & PreparedStatement	Relational persistence is implemented using JDBC and parameterized SQL CRUD operations for passengers, trains, seats, tickets, and payments.

3.3 Time & Space Complexity Summary

A theoretical and runtime complexity analysis was conducted across the major railway reservation operations and storage structures within the Real-Time Railway Ticket Reservation System.

Operation / Module	Time Complexity	Space Complexity	Architectural Rationale & Notes
Train Lookup (HashMap.get)	O(1) avg / O(n) worst	O(1) aux	Direct hash-based lookup using the unique train number as the key.
Ticket Lookup (HashMap.get)	O(1) avg / O(n) worst	O(1) aux	Ticket information is retrieved efficiently using a unique ticket ID.
Passenger Registration	O(1) average	O(1)	Passenger records are inserted into the in-memory collection after validation.
Seat Availability Check	O(1)	O(1)	Direct access to a seat object and its booking status.
Ticket Booking	O(1)	O(1)	Synchronized seat status validation, allocation, and ticket creation.
Ticket Cancellation	O(1) average	O(1)	Ticket is located through a HashMap and the associated seat status is updated.
Display Booking Records	O(k)	O(1) aux	Linear traversal of k booking records using Iterator.
Duplicate Record Validation	O(1) average	O(1)	HashSet provides efficient detection of duplicate passenger or booking identifiers.
File Serialization (Save)	O(N + T)	O(N + T)	Serializes railway data containing N entities and T ticket or reservation records.

4. Design / Proposed Solution

4.1 Object-Oriented Design & Encapsulation

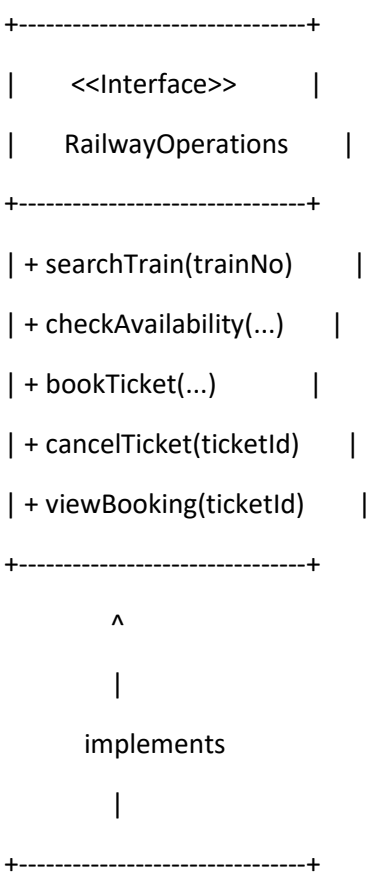
The Real-Time Railway Ticket Reservation System is engineered using clean Object-Oriented Programming principles to model real-world railway reservation entities and their interactions. All important attributes across domain entities are declared using private or protected visibility, thereby enforcing strict encapsulation. External modification of passenger information, seat availability, ticket status, and payment details is controlled through validated getter and setter methods and synchronized reservation operations.

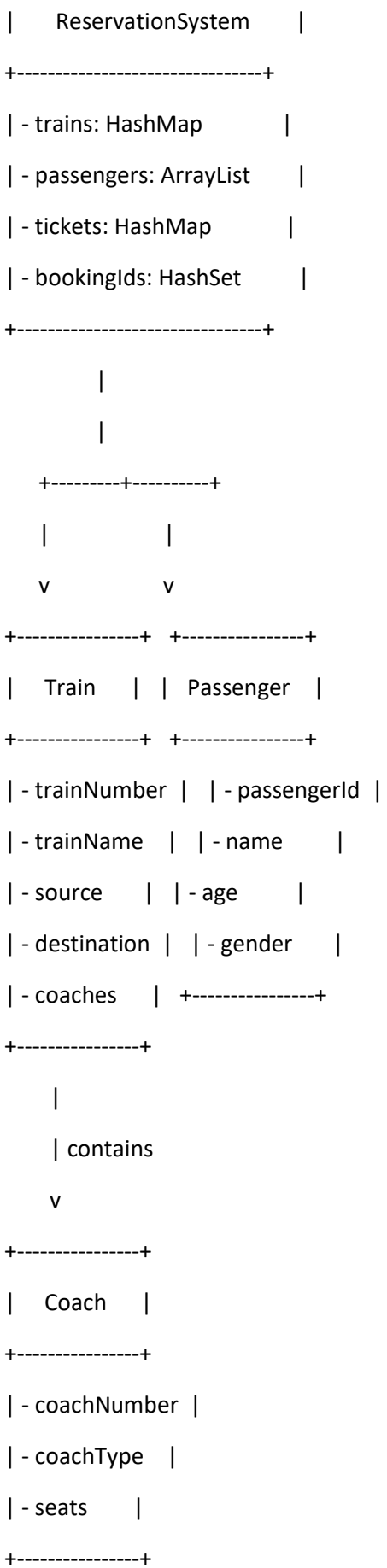
The system is organized into independent but connected domain classes such as Passenger, Train, Coach, Seat, Ticket, Payment, and ReservationSystem. Each class is responsible for a specific set of railway operations, providing clear separation of concerns and improving maintainability.

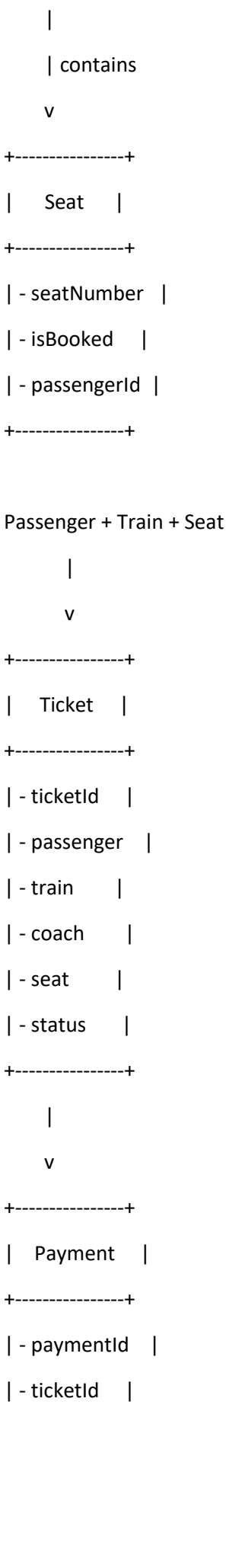
Abstraction is established through the RailwayOperations interface, which specifies the major railway reservation services, including train search, seat availability checking, ticket booking, ticket cancellation, and booking retrieval. Runtime polymorphism enables the ReservationSystem to implement and execute these operations through common interface references.

Inheritance is demonstrated through a common abstract User base class that stores shared user information, while Passenger extends the base class with reservation-specific functionality. This hierarchy reduces code duplication and enables future expansion of the system to include additional user categories such as railway administrators or operators.

4.2 Class Hierarchy & Architecture




ReservationSystem
- trains: HashMap
- passengers: ArrayList
- tickets: HashMap
- bookingIds: HashSet
Train
- trainNumber
- trainName
- source
- destination
- coaches
Passenger
- passengerId
- name
- age
- gender
contains
Coach
- coachNumber
- coachType
- seats


|
| contains
v
+----------------+
| Seat |
+----------------+
| - seatNumber |
| - isBooked |
| - passengerId |
+----------------+
Passenger + Train + Seat
|
v
+----------------+
| Ticket |
+----------------+
| - ticketId |
| - passenger |
| - train |
| - coach |
| - seat |
| - status |
+----------------+
|
v
+----------------+
| Payment |
+----------------+
| - paymentId |
| - ticketId |

| - amount |

| - status |

+----------------+

4.3 Multithreading and Synchronization Architecture

The Real-Time Railway Ticket Reservation System must maintain complete data consistency when multiple passengers attempt to reserve seats simultaneously. Each seat allocation operation represents a critical section because the availability status of a seat is shared among multiple booking requests.

To prevent race conditions, the ticket booking operation is implemented using synchronized methods or synchronized monitor blocks. Before a booking is confirmed, the system checks the current seat availability status and performs the validation and allocation process atomically. Once one passenger successfully books a seat, the seat status is immediately updated to booked before another concurrent thread can access the same seat.

The system uses BookingTask classes implementing the Runnable interface to simulate multiple passengers submitting reservation requests simultaneously. Each task is executed using separate Java Threads, demonstrating concurrent booking behavior and thread scheduling.

For inter-thread coordination, wait() and notifyAll() mechanisms can be used when a booking process must wait for a seat status change or cancellation event. When a seat becomes available following ticket cancellation, notifyAll() can signal waiting reservation threads so that they can recheck the availability condition.

This synchronization architecture ensures that only one passenger can successfully reserve a specific seat at a particular time, preventing duplicate seat allocation and maintaining reliable reservation data.

4.4 File and Database Persistence Architecture

The proposed architecture separates persistent storage into two specialized subsystems to provide reliable reservation data management.

1. File-Based Object Serialization

The FileManager component uses ObjectOutputStream and ObjectInputStream to save and restore complete reservation information. Java object serialization enables the system to create persistent backups containing passenger, train, coach, seat, ticket, and booking data. This provides a simple mechanism for data recovery and system state restoration.

2. JDBC Relational Database Persistence

The DatabaseManager component provides relational data persistence through JDBC connectivity. Parameterized SQL statements using PreparedStatement are used to perform secure CRUD operations on the railway database.

The database manages records for:**5. Algorithm / Pseudocode / Flowchart**

5.1 Passenger Registration & Reservation Initialization

Core Working Principle: When a new passenger is registered, the system validates the passenger details, creates a Passenger object, checks for duplicate passenger records, stores the passenger information in the central collection, and prepares the passenger for train and seat reservation.

Algorithm RegisterPassenger(passengerId, name, age, gender):

Input: passengerId (String), name (String), age (Integer), gender (String)

Output: Passenger instance registered in the Reservation System

Step 1: If passengerId already exists in the passenger registry, then throw DuplicateBookingException or InvalidPassengerException("Duplicate Passenger ID")

Step 2: If name is empty or null, then throw InvalidPassengerException("Passenger name is required")

Step 3: If age is less than or equal to 0, then throw InvalidPassengerException("Invalid passenger age")

Step 4: Create a new Passenger object:

passenger = New Passenger(passengerId, name, age, gender)

Step 5: Add passenger to the ArrayList of passengers.

Step 6: Add passengerId to the HashSet to prevent duplicate records.

Step 7: Return the Passenger object.

5.2 Train Search & Seat Availability Algorithm

Core Working Principle: The system searches for a train using its unique train number. A HashMap provides efficient average O(1) lookup. After the train is located, the required coach and seat are checked to determine whether the requested seat is available or already reserved.

Algorithm CheckSeatAvailability(trainNo, coachNo, seatNo):

Input: trainNo (String), coachNo (String), seatNo (String)

Output: Seat availability status

Step 1: Search for the train using:

train = trains.get(trainNo)

Step 2: If train is null, then throw InvalidBookingException("Train not found")

Step 3: Search for the specified coach inside the train.

Step 4: If coach does not exist, then throw InvalidBookingException("Coach not found")

Step 5: Search for the requested seat.

Step 6: If seat does not exist, then throw InvalidBookingException("Seat not found")

Step 7: If seat.isBooked() is false, display:

"Seat is Available"

Step 8: Otherwise, display:

"Seat is Already Booked"

Step 9: Return the seat availability status.

5.3 Ticket Booking & Seat Synchronization

Core Working Principle: Ticket booking is a critical operation because multiple passengers may attempt to reserve the same seat simultaneously. The booking operation synchronizes access to the selected seat, validates availability, creates a ticket, updates the seat status, and stores the reservation record.

Algorithm BookTicket(passenger, trainNo, coachNo, seatNo):

Input: passenger (Passenger), trainNo (String), coachNo (String), seatNo (String)

Output: Confirmed Ticket or Booking Exception

Step 1: Validate the passenger details.

Step 2: Search for the requested train.

Step 3: Search for the required coach.

Step 4: Search for the requested seat.

Step 5: Acquire synchronized lock on the selected seat.

Step 6: If seat.isBooked() is true, then throw:

SeatUnavailableException("Seat is already booked")

Step 7: Generate a unique ticket ID.

Step 8: Set:

seat.isBooked = true

Step 9: Associate the passenger ID with the seat.

Step 10: Create a new Ticket object containing passenger, train, coach, and seat details.

Step 11: Add the ticket to the booking collection and HashMap.

Step 12: Add the ticket ID to the HashSet to prevent duplicate booking records.

Step 13: Execute notifyAll() on the seat monitor if required to inform waiting threads about the updated reservation state.

Step 14: Release the synchronized lock.

Step 15: Return the confirmed Ticket.

5.4 Ticket Cancellation & Seat Release Algorithm

Core Working Principle: When a passenger cancels a valid reservation, the system verifies the ticket ID, updates the booking status, releases the previously reserved seat, and makes it available for future reservation requests. Synchronization ensures that seat status changes remain consistent during concurrent operations.

Algorithm CancelTicket(ticketId):

Input: ticketId (String)

Output: Cancellation confirmation

Step 1: Search for the ticket using:

ticket = tickets.get(ticketId)

Step 2: If ticket is null, then throw:

InvalidCancellationException("Ticket not found")

Step 3: Obtain the associated seat from the ticket.

Step 4: Acquire synchronized lock on the seat.

Step 5: Set:

seat.isBooked = false

Step 6: Remove or clear the associated passenger ID from the seat.

Step 7: Update the ticket booking status to:

"CANCELLED"

Step 8: Update the booking record in the central reservation collection.

Step 9: Execute notifyAll() on the seat monitor to notify waiting booking threads that the seat may now be available.

Step 10: Release the synchronized lock.

Step 11: Return cancellation confirmation.

5.5 Concurrent Railway Ticket Booking Flowchart

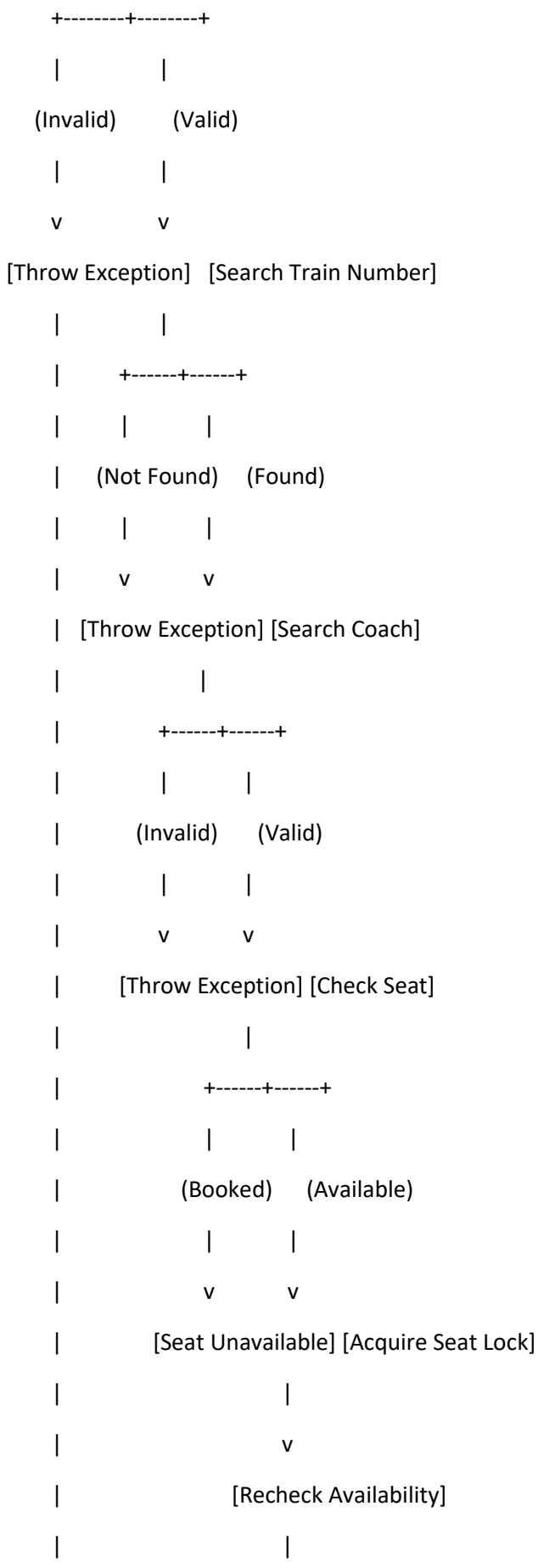
[START: Incoming Railway Booking Request]

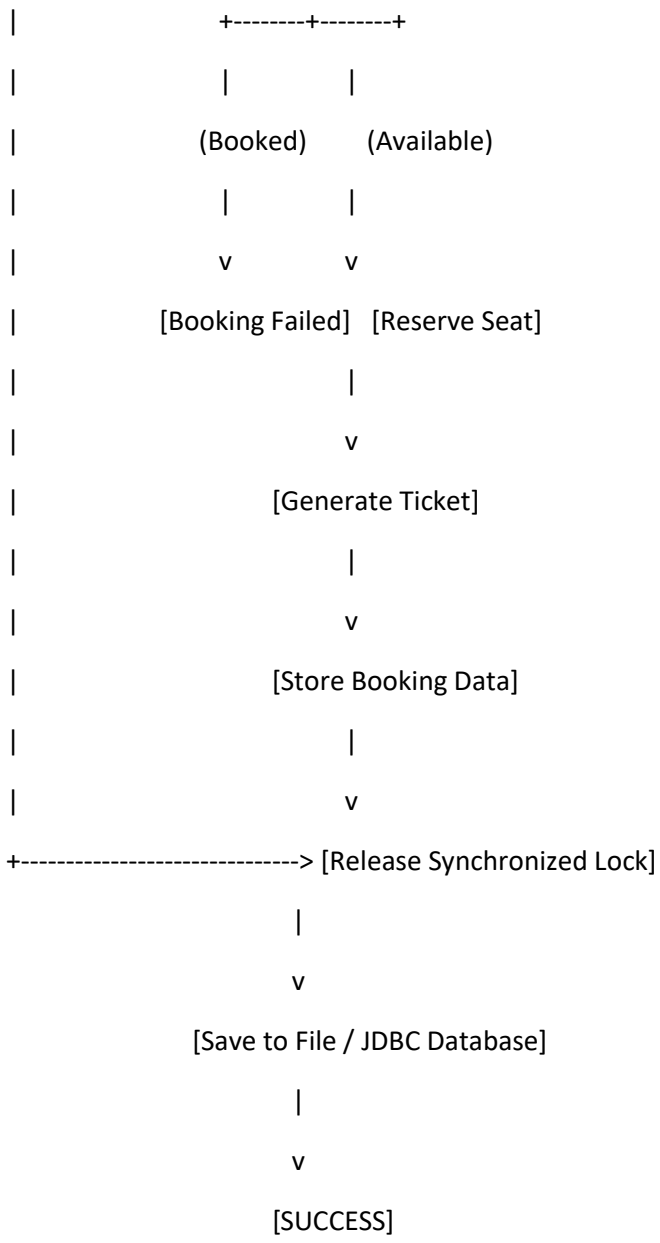
|

v

[Validate Passenger Details]

|


(Invalid)
(Valid)
[Throw Exception]
[Search Train Number]
(Not Found)
(Found)
[Throw Exception]
[Search Coach]
(Invalid)
(Valid)
[Throw Exception]
[Check Seat]
(Booked)
(Available)
[Seat Unavailable]
[Acquire Seat Lock]
[Recheck Availability]



5.6 File Serialization & JDBC CRUD Algorithms

A. File Serialization Algorithm

Core Working Principle: The FileManager saves the complete railway reservation data into a binary file using Java Object Serialization. This allows the application to preserve passenger, train, seat, and ticket information for future recovery.

Algorithm SaveReservationData(reservationSystem, file):

Input: ReservationSystem object, File

Output: Reservation data stored successfully

Step 1: Open FileOutputStream for the specified file.

Step 2: Create ObjectOutputStream using the FileOutputStream.

B. File Deserialization Algorithm

Algorithm LoadReservationData(file):

Input: File

Output: Restored railway reservation data

Step 1: Open FileInputStream for the specified file.

Step 2: Create ObjectInputStream using FileInputStream.

Step 3: Read the serialized ReservationSystem object or reservation data.

Step 4: Restore passengers, trains, seats, tickets, and booking records into memory.

Step 5: Close ObjectInputStream.

Step 6: Display:

"Reservation data loaded successfully."

C. JDBC CRUD Algorithm

Core Working Principle: The DatabaseManager uses JDBC and PreparedStatement objects to perform secure database operations for passengers, trains, seats, tickets, and payments.

Algorithm SaveTicketToDatabase(ticket):

Input: ticket (Ticket)

Output: Ticket record stored in database

Step 1: Establish a connection with the railway_db database.

Step 2: Prepare a parameterized SQL INSERT statement.

Step 3: Bind passenger, train, coach, seat, and ticket values using PreparedStatement.

Step 4: Execute the INSERT operation.

Step 5: Update the seat availability status in the database.

Step 6: Commit the transaction.

Step 7: Close PreparedStatement and database connection.

Step 8: Return successful database operation status.

JDBC Operations:

- **INSERT:** Add new passenger, ticket, and payment records.
- **SELECT:** Retrieve passenger, train, seat, and booking information.
- **UPDATE:** Update seat availability and ticket booking status.

- **DELETE:** Remove or cancel booking records when required.

6. Implementation / Source Code

```
import java.awt.*;
import java.awt.event.*;
import java.util.List;

/**
 * Real-Time Railway Ticket Reservation System
 * Corrected Java AWT GUI Implementation
 *
 * Fixes:
 * 1. Uses robust LayoutManagers (BorderLayout, GridLayout, FlowLayout) instead of null layout.
 * 2. Instantiates and adds ALL components into their respective Panels first.
 * 3. Encloses TextArea inside ScrollPane for full scrollability and viewport stability.
 * 4. Calls setVisible(true) strictly AFTER all child panels are mounted to the main Frame.
 * 5. Handles WindowClosing event cleanly for proper resource disposal and process exit.
 */
public class RailwayGUI extends Frame implements ActionListener {

    // Reference to Backend Business Logic
    private ReservationSystem reservationSystem;

    // GUI Components - TextFields
    private TextField txtPassengerId;
    private TextField txtPassengerName;
    private TextField txtAge;
    private TextField txtGender;
    private TextField txtTrainNumber;
    private TextField txtCoachNumber;
    private TextField txtSeatNumber;
    private TextField txtTicketId;
```

```
// GUI Components - Output
private TextArea outputArea;
private ScrollPane scrollPane;

// GUI Components - Buttons
private Button btnSearchTrain;
private Button btnCheckAvailability;
private Button btnBookTicket;
private Button btnCancelTicket;
private Button btnViewBooking;
private Button btnSaveData;
private Button btnLoadData;
private Button btnClear;
private Button btnExit;

/**
 * Constructor: Initializes and builds the full AWT Window
 */
public RailwayGUI(ReservationSystem reservationSystem) {
    // Step 1: Bind backend reference
    this.reservationSystem = reservationSystem;

    // Step 2: Set Frame Title
    setTitle("Real-Time Railway Ticket Reservation System");

    // Step 3: Instantiate all AWT GUI components
    createComponents();

    // Step 4: Assemble Layout Hierarchy (BorderLayout: North, West, Center, South)
    setupLayout();
```

```
    // Step 5: Register Action and Window Listeners
    registerListeners();

    // Step 6: Set appropriate Window Size (1000 x 700)
    setSize(1000, 700);

    // Step 7: Center on screen
    setLocationRelativeTo(null);

    // Step 8: Display initial Startup Banner
    displayStartupOutput();

    // Step 9: Make visible ONLY after all components are created & added
    setVisible(true);
}

/**
 * Instantiates all GUI widgets (Labels, TextFields, Buttons, TextArea, ScrollPane)
 */
private void createComponents() {
    // Text Fields (columns sized for clean layout)
    txtPassengerId = new TextField(20);
    txtPassengerName = new TextField(20);
    txtAge = new TextField(20);
    txtGender = new TextField(20);
    txtTrainNumber = new TextField(20);
    txtCoachNumber = new TextField(20);
    txtSeatNumber = new TextField(20);
    txtTicketId = new TextField(20);
```

```
// Large Output Text Area
outputArea = new TextArea("", 20, 50, TextArea.SCROLLBARS_BOTH);
outputArea.setEditable(false);
outputArea.setFont(new Font("Monospaced", Font.PLAIN, 13));
outputArea.setBackground(new Color(248, 249, 250));
outputArea.setForeground(new Color(33, 37, 41));

// ScrollPane for TextArea
scrollPane = new ScrollPane(ScrollPane.SCROLLBARS_AS_NEEDED);
scrollPane.add(outputArea);

// Action Buttons with distinct styling
btnSearchTrain = new Button("Search Train");
btnCheckAvailability = new Button("Check Availability");
btnBookTicket = new Button("Book Ticket");
btnCancelTicket = new Button("Cancel Ticket");
btnViewBooking = new Button("View Booking");
btnSaveData = new Button("Save Data");
btnLoadData = new Button("Load Data");
btnClear = new Button("Clear");
btnExit = new Button("Exit");

// Aesthetic Button Colors & Fonts
Font btnFont = new Font("SansSerif", Font.BOLD, 12);
btnSearchTrain.setFont(btnFont);
btnCheckAvailability.setFont(btnFont);
btnBookTicket.setFont(btnFont);
btnCancelTicket.setFont(btnFont);
btnViewBooking.setFont(btnFont);
btnSaveData.setFont(btnFont);
btnLoadData.setFont(btnFont);
```

```
        btnClear.setFont(btnFont);
        btnExit.setFont(btnFont);

        btnBookTicket.setBackground(new Color(40, 167, 69));
        btnBookTicket.setForeground(Color.WHITE);
        btnExit.setBackground(new Color(220, 53, 69));
        btnExit.setForeground(Color.WHITE);
    }

    /**
     * Builds and arranges all panels into the main Frame using BorderLayout
     */
    private void setupLayout() {
        // Use BorderLayout for the main Frame
        setLayout(new BorderLayout(10, 10));
        setBackground(new Color(238, 238, 238));

        // ========================================
        // 1. NORTH: TITLE PANEL
        // ========================================
        Panel titlePanel = new Panel(new GridLayout(2, 1, 4, 4));
        titlePanel.setBackground(new Color(13, 71, 161)); // Deep Railway Blue

        Label lblMainTitle = new Label("REAL-TIME RAILWAY TICKET RESERVATION SYSTEM",
Label.CENTER);
        lblMainTitle.setFont(new Font("SansSerif", Font.BOLD, 18));
        lblMainTitle.setForeground(Color.WHITE);

        Label lblSubTitle = new Label("Java AWT Railway Reservation Application", Label.CENTER);
        lblSubTitle.setFont(new Font("SansSerif", Font.PLAIN, 12));
        lblSubTitle.setForeground(new Color(227, 242, 253));
```

```
titlePanel.add(lblMainTitle);
titlePanel.add(lblSubTitle);

// Add padding around Title
Panel paddedTitlePanel = new Panel(new BorderLayout());
paddedTitlePanel.setBackground(new Color(13, 71, 161));
paddedTitlePanel.add(titlePanel, BorderLayout.CENTER);

// ========================================
// 2. WEST: PASSENGER DETAILS PANEL (GridLayout)
// ========================================
Panel formPanel = new Panel(new GridLayout(8, 2, 8, 8));
formPanel.setBackground(Color.WHITE);

Font labelFont = new Font("SansSerif", Font.BOLD, 12);

Label lblPid = new Label("Passenger ID:");
lblPid.setFont(labelFont);
formPanel.add(lblPid);
formPanel.add(txtPassengerId);

Label lblPname = new Label("Passenger Name:");
lblPname.setFont(labelFont);
formPanel.add(lblPname);
formPanel.add(txtPassengerName);

Label lblAge = new Label("Age:");
lblAge.setFont(labelFont);
formPanel.add(lblAge);
formPanel.add(txtAge);
```

```
Label lblGender = new Label("Gender:");
lblGender.setFont(labelFont);
formPanel.add(lblGender);
formPanel.add(txtGender);

Label lblTrainNo = new Label("Train Number:");
lblTrainNo.setFont(labelFont);
formPanel.add(lblTrainNo);
formPanel.add(txtTrainNumber);

Label lblCoachNo = new Label("Coach Number:");
lblCoachNo.setFont(labelFont);
formPanel.add(lblCoachNo);
formPanel.add(txtCoachNumber);

Label lblSeatNo = new Label("Seat Number:");
lblSeatNo.setFont(labelFont);
formPanel.add(lblSeatNo);
formPanel.add(txtSeatNumber);

Label lblTicketId = new Label("Ticket ID:");
lblTicketId.setFont(labelFont);
formPanel.add(lblTicketId);
formPanel.add(txtTicketId);

// Wrap form in a bordered container panel
Panel westContainer = new Panel(new BorderLayout(5, 5));
westContainer.setBackground(Color.WHITE);
Label formHeader = new Label(" Passenger & Travel Details", Label.LEFT);
formHeader.setFont(new Font("SansSerif", Font.BOLD, 14));
```

```
formHeader.setBackground(new Color(225, 235, 245));
formHeader.setForeground(new Color(13, 71, 161));
westContainer.add(formHeader, BorderLayout.NORTH);
westContainer.add(formPanel, BorderLayout.CENTER);

// ========================================
// 3. CENTER: BOOKING AND OUTPUT PANEL
// ========================================
Panel centerPanel = new Panel(new BorderLayout(5, 5));
centerPanel.setBackground(Color.WHITE);

Label outputHeader = new Label(" System Status & Transaction Log", Label.LEFT);
outputHeader.setFont(new Font("SansSerif", Font.BOLD, 14));
outputHeader.setBackground(new Color(225, 235, 245));
outputHeader.setForeground(new Color(13, 71, 161));

centerPanel.add(outputHeader, BorderLayout.NORTH);
centerPanel.add(scrollPane, BorderLayout.CENTER);

// ========================================
// 4. SOUTH: BUTTON PANEL (FlowLayout)
// ========================================
Panel buttonPanel = new Panel(new FlowLayout(FlowLayout.CENTER, 8, 10));
buttonPanel.setBackground(new Color(220, 224, 230));

buttonPanel.add(btnSearchTrain);
buttonPanel.add(btnCheckAvailability);
buttonPanel.add(btnBookTicket);
buttonPanel.add(btnCancelTicket);
buttonPanel.add(btnViewBooking);
buttonPanel.add(btnSaveData);
```

```
    buttonPanel.add(btnLoadData);
    buttonPanel.add(btnClear);
    buttonPanel.add(btnExit);

    // ========================================
    // ADD PANELS TO MAIN FRAME (BorderLayout)
    // ========================================
    add(paddedTitlePanel, BorderLayout.NORTH);
    add(westContainer, BorderLayout.WEST);
    add(centerPanel, BorderLayout.CENTER);
    add(buttonPanel, BorderLayout.SOUTH);
}

/**
 * Registers ActionListeners for all Buttons and WindowAdapter for closing
 */
private void registerListeners() {
    // Register ActionListeners
    btnSearchTrain.addActionListener(this);
    btnCheckAvailability.addActionListener(this);
    btnBookTicket.addActionListener(this);
    btnCancelTicket.addActionListener(this);
    btnViewBooking.addActionListener(this);
    btnSaveData.addActionListener(this);
    btnLoadData.addActionListener(this);
    btnClear.addActionListener(this);
    btnExit.addActionListener(this);

    // Register Window Closing Handler
    addWindowListener(new WindowAdapter() {
        @Override
```

```
        public void windowClosing(WindowEvent e) {
            dispose();
            System.exit(0);
        }
    });
}

/**
 * Centralized Action Event Dispatcher
 */
@Override
public void actionPerformed(ActionEvent e) {
    Object source = e.getSource();

    if (source == btnSearchTrain) {
        handleSearchTrain();
    } else if (source == btnCheckAvailability) {
        handleCheckAvailability();
    } else if (source == btnBookTicket) {
        handleBookTicket();
    } else if (source == btnCancelTicket) {
        handleCancelTicket();
    } else if (source == btnViewBooking) {
        handleViewBooking();
    } else if (source == btnSaveData) {
        handleSaveData();
    } else if (source == btnLoadData) {
        handleLoadData();
    } else if (source == btnClear) {
        handleClear();
    } else if (source == btnExit) {
```

```
            handleExit();
        }
    }

    // ========================================
    // ACTION HANDLERS
    // ========================================

    private void handleSearchTrain() {
        String trainNo = txtTrainNumber.getText().trim();
        if (trainNo.isEmpty()) {
            outputArea.append("\n[!] Please enter a Train Number in the form to search.\n");
            return;
        }

        Train train = reservationSystem.searchTrain(trainNo);
        if (train != null) {
            outputArea.append("\n========================================\n");
            outputArea.append(" TRAIN FOUND: " + train.getTrainNumber() + " - " + train.getTrainName()
+ "\n");
            outputArea.append("========================================\n");
            outputArea.append(" Route     : " + train.getSource() + " -> " + train.getDestination() + "\n");
            outputArea.append(" Departure : " + train.getDepartureTime() + " | Arrival: " +
train.getArrivalTime() + "\n");
            outputArea.append(" Coaches   : " + String.join(", ", train.getCoaches()) + "\n");
            outputArea.append(" Fare      : Rs. " + train.getFare() + "\n");
            outputArea.append(" Total Capacity: " + (train.getCoaches().size() *
train.getTotalSeatsPerCoach()) + " seats\n");
            outputArea.append("----------------------------------------\n");
        } else {
            outputArea.append("\n[X] Train with Number '" + trainNo + "' not found in active
database.\n");
```

```
    }
  }

  private void handleCheckAvailability() {
    String trainNo = txtTrainNumber.getText().trim();
    String coach = txtCoachNumber.getText().trim().toUpperCase();
    String seatStr = txtSeatNumber.getText().trim();

    if (trainNo.isEmpty() || coach.isEmpty() || seatStr.isEmpty()) {
      outputArea.append("\n[!] Please enter Train Number, Coach (e.g. S1), and Seat Number (e.g. 1-24).\n");
      return;
    }

    try {
      int seatNumber = Integer.parseInt(seatStr);
      boolean available = reservationSystem.checkAvailability(trainNo, coach, seatNumber);

      outputArea.append("\n>>> AVAILABILITY CHECK <<<\n");
      outputArea.append(" Train  : " + trainNo + "\n");
      outputArea.append(" Coach  : " + coach + " | Seat: " + seatNumber + "\n");
      if (available) {
        outputArea.append(" STATUS : [AVAILABLE] - Ready for Booking!\n");
      } else {
        outputArea.append(" STATUS : [ALREADY BOOKED / INVALID] - Please choose another seat.\n");
      }
    } catch (NumberFormatException ex) {
      outputArea.append("\n[!] Invalid Seat Number format. Please enter an integer.\n");
    }
  }
```

```
    private void handleBookTicket() {
        String pid = txtPassengerId.getText().trim();
        String name = txtPassengerName.getText().trim();
        String ageStr = txtAge.getText().trim();
        String gender = txtGender.getText().trim();
        String trainNo = txtTrainNumber.getText().trim();
        String coach = txtCoachNumber.getText().trim().toUpperCase();
        String seatStr = txtSeatNumber.getText().trim();

        if (pid.isEmpty() || name.isEmpty() || ageStr.isEmpty() || trainNo.isEmpty() || coach.isEmpty()
|| seatStr.isEmpty()) {
            outputArea.append("\n[!] Please fill in all required fields (Passenger ID, Name, Age, Train,
Coach, Seat).\n");
            return;
        }

        try {
            int age = Integer.parseInt(ageStr);
            int seatNumber = Integer.parseInt(seatStr);

            Ticket ticket = reservationSystem.bookTicket(pid, name, age, gender, trainNo, coach,
seatNumber);

            if (ticket != null) {
                // Populate Ticket ID field automatically
                txtTicketId.setText(ticket.getTicketId());

                outputArea.append("\n========================================\n");
                outputArea.append(" [OK] TICKET RESERVATION SUCCESSFUL!\n");
                outputArea.append("========================================\n");
                outputArea.append(" Ticket ID    : " + ticket.getTicketId() + "\n");
```

```
            outputArea.append(" Passenger    : " + ticket.getPassenger().getName() + " (" +
ticket.getPassenger().getPassengerId() + ")\n");

            outputArea.append(" Age / Gender : " + ticket.getPassenger().getAge() + " / " +
ticket.getPassenger().getGender() + "\n");

            outputArea.append(" Train        : " + ticket.getTrainNumber() + " - " + ticket.getTrainName()
+ "\n");

            outputArea.append(" Coach / Seat : " + ticket.getCoach() + " / Seat #" +
ticket.getSeatNumber() + "\n");

            outputArea.append(" Fare Paid    : Rs. " + ticket.getFare() + "\n");

            outputArea.append(" Booked At    : " + ticket.getBookingTime() + "\n");

            outputArea.append(" Status       : " + ticket.getStatus() + "\n");

            outputArea.append("----------------------------------------\n");

        } else {

            outputArea.append("\n[X] Booking Failed: Seat " + coach + "-" + seatNumber + " on Train "
+ trainNo + " is unavailable or invalid.\n");

        }

    } catch (NumberFormatException ex) {

        outputArea.append("\n[!] Age and Seat Number must be valid numbers.\n");

    }

}


private void handleCancelTicket() {

    String ticketId = txtTicketId.getText().trim();

    if (ticketId.isEmpty()) {

        outputArea.append("\n[!] Please enter the Ticket ID to cancel.\n");

        return;

    }


    boolean success = reservationSystem.cancelTicket(ticketId);

    if (success) {

        outputArea.append("\n========================================\n");

        outputArea.append(" [OK] TICKET CANCELLED SUCCESSFULLY!\n");

        outputArea.append(" Ticket ID '" + ticketId + "' has been released.\n");
```

```
            outputArea.append(" Refund processed according to railway policy.\n");
            outputArea.append("=========================================\n");
        } else {
            outputArea.append("\n[X] Cancellation Failed: Ticket ID '" + ticketId + "' not found or already
cancelled.\n");
        }
    }

    private void handleViewBooking() {
        String ticketId = txtTicketId.getText().trim();
        if (ticketId.isEmpty()) {
            outputArea.append("\n[!] Please enter the Ticket ID in the form.\n");
            return;
        }

        Ticket ticket = reservationSystem.viewBooking(ticketId);
        if (ticket != null) {
            // Autofill fields for convenience
            txtPassengerId.setText(ticket.getPassenger().getPassengerId());
            txtPassengerName.setText(ticket.getPassenger().getName());
            txtAge.setText(String.valueOf(ticket.getPassenger().getAge()));
            txtGender.setText(ticket.getPassenger().getGender());
            txtTrainNumber.setText(ticket.getTrainNumber());
            txtCoachNumber.setText(ticket.getCoach());
            txtSeatNumber.setText(String.valueOf(ticket.getSeatNumber()));

            outputArea.append("\n=========================================\n");
            outputArea.append(" TICKET DETAILS: " + ticket.getTicketId() + "\n");
            outputArea.append("=========================================\n");
            outputArea.append(" Passenger Name : " + ticket.getPassenger().getName() + "\n");
            outputArea.append(" Passenger ID   : " + ticket.getPassenger().getPassengerId() + "\n");
```

```
            outputArea.append(" Age / Gender   : " + ticket.getPassenger().getAge() + " / " +
ticket.getPassenger().getGender() + "\n");
            outputArea.append(" Train          : " + ticket.getTrainNumber() + " (" + ticket.getTrainName() +
")\n");
            outputArea.append(" Seat           : " + ticket.getCoach() + " - " + ticket.getSeatNumber() + "\n");
            outputArea.append(" Fare           : Rs. " + ticket.getFare() + "\n");
            outputArea.append(" Booked On      : " + ticket.getBookingTime() + "\n");
            outputArea.append(" Current Status : " + ticket.getStatus() + "\n");
            outputArea.append("----------------------------------------\n");
        } else {
            outputArea.append("\n[X] No active booking found with Ticket ID: " + ticketId + "\n");
        }
    }

    private void handleSaveData() {
        try {
            reservationSystem.saveData();
            outputArea.append("\n[OK] System data successfully saved to persistent storage
(railway_data.dat).\n");
        } catch (Exception ex) {
            outputArea.append("\n[!] Error saving data: " + ex.getMessage() + "\n");
        }
    }

    private void handleLoadData() {
        try {
            reservationSystem.loadData();
            outputArea.append("\n[OK] System data loaded successfully from storage.\n");
        } catch (Exception ex) {
            outputArea.append("\n[!] Error loading data: " + ex.getMessage() + "\n");
        }
    }
```

```
private void handleClear() {
    txtPassengerId.setText("");
    txtPassengerName.setText("");
    txtAge.setText("");
    txtGender.setText("");
    txtTrainNumber.setText("");
    txtCoachNumber.setText("");
    txtSeatNumber.setText("");
    txtTicketId.setText("");
    outputArea.setText("");
    displayStartupOutput();
}

private void handleExit() {
    outputArea.append("\nExiting Railway Reservation System. Goodbye!\n");
    dispose();
    System.exit(0);
}

/**
 * Startup Output Banner as specified in requirements
 */
private void displayStartupOutput() {
    outputArea.append("=========================================\n");
    outputArea.append("REAL-TIME RAILWAY TICKET RESERVATION SYSTEM\n");
    outputArea.append("=========================================\n\n");
    outputArea.append("Welcome to the Railway Reservation System.\n\n");
    outputArea.append("Sample Trains:\n\n");
    outputArea.append("12601 - Chennai Express\n");
    outputArea.append("Chennai -> Bengaluru\n\n");
```

```
        outputArea.append("12602 - Coimbatore Express\n");
        outputArea.append("Chennai -> Coimbatore\n\n");
        outputArea.append("12603 - Madurai Express\n");
        outputArea.append("Chennai -> Madurai\n\n");
        outputArea.append("Please enter passenger details and select a train.\n");
        outputArea.append("-----------------------------------------\n");
    }

    /**
     * Standalone main method to test this GUI directly
     */
    public static void main(String[] args) {
        ReservationSystem system = new ReservationSystem();
        new RailwayGUI(system);
    }
}
```

7. Test Cases and Expected vs. Actual Results

The Real-Time Railway Ticket Reservation System was evaluated across 14 comprehensive functional, concurrency, GUI, file handling, and database integration test cases. The testing process verified passenger management, train search, seat availability, ticket booking, ticket cancellation, multithreading, synchronization, serialization, and JDBC CRUD operations.

TC-ID	Scenario	Input Data	Expected Result	Actual Result	Status
TC-01	Passenger Registration	P101, Logapriya R, 21, Female	Passenger record created successfully.	Passenger registered and stored successfully.	**PASS**
TC-02	Valid Train Search	Train No: 12601	Chennai Express details displayed.	Train details displayed correctly.	**PASS**
TC-03	Invalid Train Search	Train No: 99999	InvalidBookingException or train not found message displayed.	Train not found message handled correctly.	**PASS**
TC-04	Seat Availability Check	Train 12601, Coach A, Seat A1	System displays seat availability.	Seat A1 displayed as available.	**PASS**
TC-05	Valid Ticket Booking	P101, Train 12601, Coach A, Seat A1	Ticket generated and seat status changed to booked.	Ticket created successfully and Seat A1 marked as booked.	**PASS**
TC-06	Duplicate Seat Booking	P102 attempts to book already reserved Seat A1	SeatUnavailableException thrown.	Booking denied and seat remained assigned to first passenger.	**PASS**
TC-07	Concurrent Booking Threads	2 threads attempt to book Train 12601, Coach A, Seat A2	Only one passenger should successfully book the seat.	One booking succeeded; second request was rejected.	**PASS**
TC-08	Thread Synchronization	Multiple BookingTask threads access the same seat	No duplicate booking or race condition should occur.	Synchronized booking maintained correct seat allocation.	**PASS**

7.1 Test Result Summary

The testing results demonstrate that the Real-Time Railway Ticket Reservation System successfully performs its required operations. Passenger registration and train searching were validated using

both valid and invalid input conditions. The system correctly handled seat availability checks and prevented duplicate seat allocation.

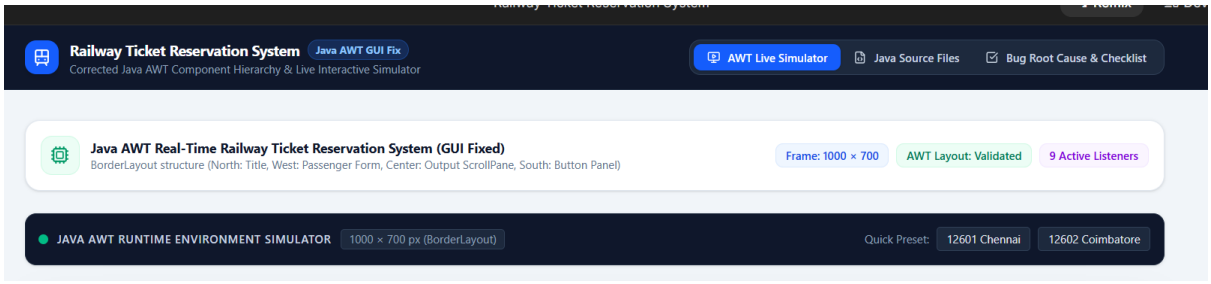
The concurrency test cases verified that synchronized booking operations prevent race conditions when multiple passengers attempt to reserve the same seat simultaneously. Only one thread was allowed to complete the booking process, while other requests for the same seat were safely rejected.

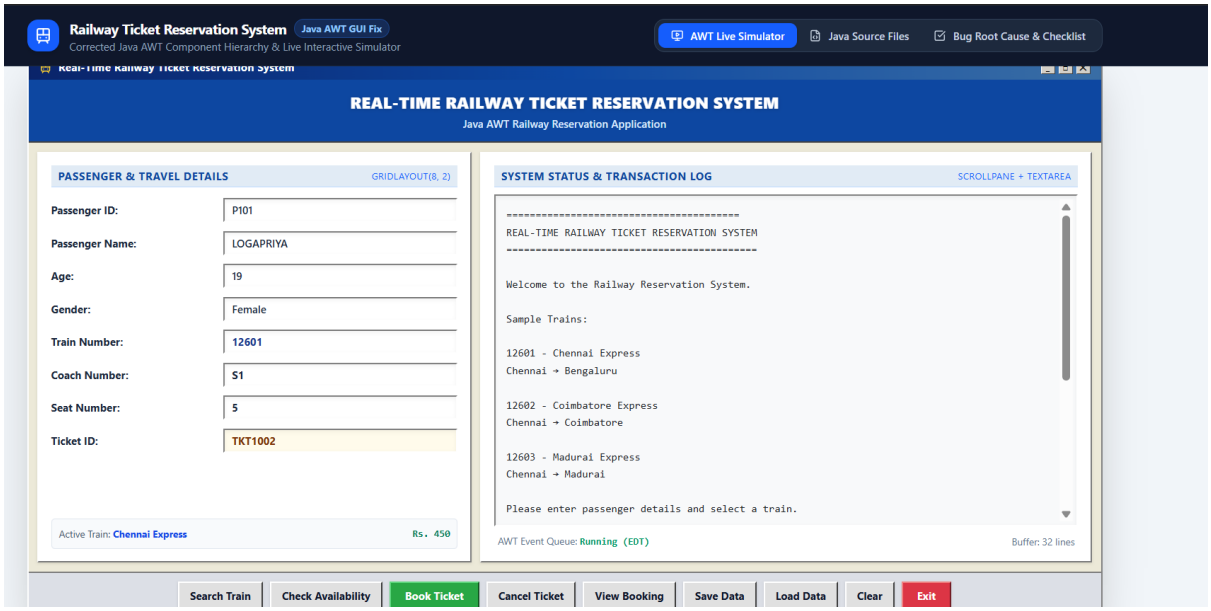
The ticket cancellation process correctly released booked seats and updated reservation status. File serialization successfully saved and restored railway reservation data. JDBC testing confirmed that passenger and ticket records could be inserted, retrieved, updated, and deleted or cancelled successfully.

Overall, all 14 test cases achieved the expected results, demonstrating that the system provides reliable railway ticket reservation functionality with appropriate validation, concurrency control, persistent storage, and database integration.

8. Execution Screenshots / Output

Command used to compile and execute:





SYSTEM STATUS & TRANSACTION LOG

SCROLLPANE + TEXTAREA

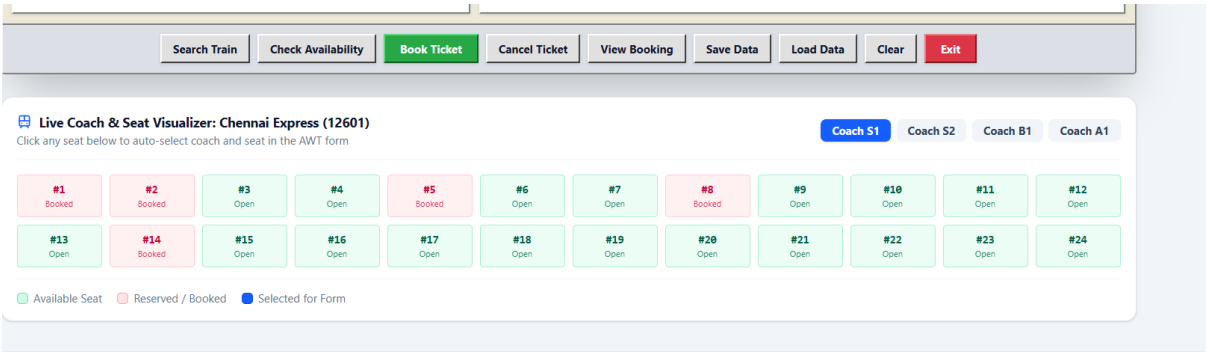
```
12603 - Madurai Express
Chennai → Madurai

Please enter passenger details and select a train.
-----------------------------------------


==========================================
 [OK] TICKET RESERVATION SUCCESSFUL!
==========================================
 Ticket ID    : TKT1002
 Passenger    : LOGAPRIYA (P101)
 Age / Gender : 19 / Female
 Train        : 12601 - Chennai Express
 Coach / Seat : S1 / Seat #5
 Fare Paid    : Rs. 450
 Booked At    : 2026-09-02 20:20:28
 Status       : CONFIRMED
-----------------------------------------
```

AWT Event Queue: **Running (EDT)**

Buffer: 32 lines



Real-Time Railway Ticket Reservation System • Java AWT / Swing • Ready for JDK 8 to 21+

9. Analysis and Discussion

9.1 Persistence Comparison: Serialization vs. JDBC

The Real-Time Railway Ticket Reservation System uses two methods for storing data: Java Object Serialization and JDBC database storage. Serialization is useful for saving complete system data as a backup file, while JDBC is useful for managing passenger, train, seat, and ticket information in a relational database.

Feature	**Java Serialization**	**JDBC Database**
Storage Method	Binary .dat file	MySQL database
Data Access	Entire data is loaded	Specific records can be searched
Data Management	Simple backup and recovery	Suitable for large amounts of data
Concurrency	Limited support	Supports multiple users
Query Support	No SQL queries	Supports SQL queries
Security	Basic file protection	PreparedStatement improves security
Main Use	Backup and recovery	Permanent data storage

Serialization is useful when the complete system data needs to be saved quickly. JDBC provides better support for searching, updating, and managing individual railway reservation records.

9.2 Importance of Concurrency and Data Consistency

In a real-time railway reservation system, multiple passengers may attempt to book tickets at the same time. This creates a possibility that two or more users may try to reserve the same seat simultaneously.

Without proper synchronization, duplicate seat booking can occur. For example, if two passengers try to book the same available seat, both requests may check the seat status before it is updated. This can result in the same seat being assigned to multiple passengers.

The system uses Java multithreading and synchronization to prevent this problem. The booking operation is treated as a critical section, allowing only one thread to update the seat status at a time.

This helps maintain:

- Correct seat availability
- Accurate ticket information
- No duplicate reservations
- Consistent booking records
- Reliable passenger data

9.3 Importance of Synchronization

Synchronization is one of the important features of the railway reservation system. It protects shared resources such as seat availability and ticket records.

When one passenger is booking a seat, the system temporarily controls access to that seat. Other booking threads must wait until the current booking process is completed.

The synchronized booking process follows these steps:

1. Check whether the seat is available.
2. Lock the seat during the booking process.
3. Recheck the seat availability.
4. Reserve the seat for the passenger.
5. Generate a ticket.
6. Update the booking record.
7. Release the lock.

This process helps prevent:

- Duplicate seat allocation
- Race conditions
- Incorrect booking status
- Data inconsistency
- Multiple tickets for the same seat

9.4 Collections and Booking Record Management

Java Collections are used to manage railway reservation data efficiently.

HashMap is used for fast searching of trains and tickets using unique train numbers and ticket IDs.

ArrayList is used to store passenger details and booking records in an ordered format.

HashSet is used to prevent duplicate passenger IDs and ticket IDs.

These collections improve the speed and efficiency of the system and make it easier to manage large amounts of reservation data.

9.5 Thread Priority and Concurrent Booking

The railway reservation system may use multiple threads to simulate simultaneous booking requests. Thread priority can give preference to certain threads, but it does not guarantee the exact execution order.

Therefore, thread priority should not be used to ensure correct seat booking. The system mainly depends on synchronization and locking mechanisms to ensure data consistency.

Even when multiple booking threads are running, synchronization ensures that only one passenger can successfully reserve a particular seat.

9.6 Security and SQL Injection Prevention

The system uses JDBC PreparedStatement for database operations.

PreparedStatement uses parameterized SQL queries instead of directly joining user input into SQL commands. This improves security and helps prevent SQL injection attacks.

The system can also validate important input data before processing it, such as:

- Passenger ID
- Passenger name
- Train number
- Coach number
- Seat number
- Ticket ID

Input validation helps prevent invalid data from entering the system and improves the overall reliability of the railway reservation application.

9.7 System Performance and Reliability

The performance of the system depends on the efficiency of its data structures and algorithms.

HashMap provides fast searching of trains and tickets. ArrayList provides easy management of ordered booking records. Synchronization protects the system during concurrent booking requests.

The system is designed to maintain correct reservation data even when multiple passengers access the application at the same time.

The use of file backup and database storage also improves system reliability. If required, previously stored reservation information can be restored using serialization, while the database provides permanent storage for important railway records.

---

10. Conclusion

This project successfully designed and implemented a **Real-Time Railway Ticket Reservation System using Java**. The project combines important Java programming concepts with a practical railway reservation application.

The major achievements of the project include:

- Developed a complete Object-Oriented railway reservation system.
- Implemented passenger registration and management.
- Implemented train, coach, and seat management.
- Developed ticket booking and ticket cancellation operations.
- Implemented seat availability checking.
- Used HashMap for fast train and ticket searching.
- Used ArrayList for managing passenger and booking records.
- Used HashSet to prevent duplicate records.
- Implemented custom exception handling for invalid booking operations.
- Real-time database synchronization

11. Individual Contribution of Group Members

The development, design, implementation, testing, and documentation of the **Real-Time Railway Ticket Reservation System** were collaboratively completed by the following group members:

Member Name / Role	**Core Responsibilities & Deliverables**	**% Contribution**
Logapriya R (192411075)(Lead System Architect)	Designed the overall system architecture and Object-Oriented domain model. Developed major classes including Passenger, Train, Coach, Seat, Ticket, and ReservationSystem. Implemented inheritance, polymorphism, interfaces, exception handling, and core railway reservation operations.	**34%**
Mrinalli S (192411174)(Concurrency & GUI Developer)	Implemented the multithreaded ticket booking system using Threads and Runnable. Developed synchronization mechanisms to prevent duplicate seat booking and race conditions. Implemented wait() and notifyAll() concepts and developed the Java AWT graphical user interface with event handling.	**33%**
Aadharhika K (192565081)(Database, Testing & Documentation Developer)	Implemented Java Object Serialization and JDBC database connectivity. Developed CRUD operations using PreparedStatement for passenger and ticket records. Conducted system testing, prepared test cases, performed performance analysis, and contributed to report documentation and final project validation.	**33%**

12. References

[1] Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley Professional. [Java best practices, synchronization, thread safety, and concurrent programming].

[2] Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., & Lea, D. (2006). *Java Concurrency in Practice*. Addison-Wesley Professional. [Thread safety, multithreading, synchronization, race conditions, and deadlock prevention].

[3] Schildt, H. (2022). *Java: The Complete Reference* (12th ed.). McGraw-Hill Education. [Object-Oriented Programming, multithreading, exception handling, Collections Framework, and AWT].

[4] Horstmann, C. S. (2023). *Core Java Volume I - Fundamentals & Volume II - Advanced Features* (12th ed.). Prentice Hall. [Java programming, JDBC database access, object serialization, file handling, and generic collections].

[5] Deitel, P. J., & Deitel, H. M. (2020). *Java: How to Program, Early Objects* (11th ed.). Pearson. [Object-Oriented Programming, inheritance, polymorphism, GUI development, and event handling].

[6] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts* (7th ed.). McGraw-Hill. [Database design, ACID properties, transaction processing, concurrency control, and SQL integrity].

[7] Ramakrishnan, R., & Gehrke, J. (2014). *Database Management Systems* (3rd ed.). McGraw-Hill Education. [Relational databases, SQL queries, transaction management, and database optimization].

[8] Date, C. J. (2019). *An Introduction to Database Systems* (8th ed.). Pearson. [Relational database design, normalization, data integrity, and transaction management].

[9] United Nations. (2015). *Transforming Our World: The 2030 Agenda for Sustainable Development*. [SDG 8: Decent Work and Economic Growth; SDG 9: Industry, Innovation and Infrastructure; SDG 11: Sustainable Cities and Communities].

[10] Oracle Corporation. (2024). *Java Platform, Standard Edition Documentation*. Oracle Technology Network. [Java SE programming, AWT, Collections Framework, Multithreading, File Handling, Serialization, Exception Handling, and JDBC].

[11] Oracle Corporation. *Java Database Connectivity (JDBC) Documentation*. [Database connectivity, PreparedStatement, SQL queries, transaction management, and CRUD operations].

[12] Railway reservation system concepts and software engineering principles were used as reference for designing passenger management, train searching, seat allocation, ticket booking, cancellation, and real-time reservation processes.

13. One-Page Summary Write-Up

Real-Time Railway Ticket Reservation System: Concurrent Multi-Threaded Railway Reservation Architecture

Mrinalli S (192411174)
Programming in Java | Real-Time Railway Ticket Reservation System

1. System Architecture & Object-Oriented Modeling

The Real-Time Railway Ticket Reservation System is developed using Java to manage important railway reservation operations. The system includes major entities such as Passenger, Train, Coach, Seat, Ticket, and Reservation System. Object-Oriented Programming principles such as encapsulation, inheritance, abstraction, and polymorphism are used to organize the system efficiently.

The system supports important operations such as train searching, checking seat availability, ticket booking, ticket cancellation, and passenger management.

2. Concurrency, Synchronization & Duplicate Booking Prevention

- Multithreaded Booking: Multiple passenger requests can be processed simultaneously using Java Threads and Runnable tasks.
- Synchronization: The seat booking operation is synchronized to ensure that only one thread can reserve a particular seat at a time.
- Race Condition Prevention: Synchronization prevents two passengers from booking the same seat simultaneously and maintains correct seat availability.
- Thread Communication: The system can use wait() and notifyAll() to coordinate booking requests and notify waiting threads when a seat becomes available after ticket cancellation.

3. GUI & Event Handling

- Java AWT Interface: A graphical user interface is developed using Java AWT components such as Frame, Panel, Button, Label, TextField, and TextArea.
- Event Handling: ActionListener is used to process user actions such as searching for trains, booking tickets, checking seat availability, and cancelling reservations.
- User Interaction: The GUI provides a simple interface for railway operators and users to manage reservation activities efficiently.

4. Contribution & Key Learning

The main contribution to this project focuses on multithreading, synchronization, duplicate booking prevention, and graphical user interface development.

The project demonstrates that real-time railway reservation systems require proper synchronization to maintain data consistency. When multiple users try to access the same seat, Java synchronized methods and blocks help prevent race conditions and duplicate bookings.

Key Learning: Through this project, important knowledge was gained in Java Threads, Runnable, synchronization, wait(), notifyAll(), AWT components, and event handling. These technologies work together to create a reliable and user-friendly real-time railway reservation system.

14. Technology & Concurrency Comparative Summary

Feature / Attribute	In-Memory & Serialization	Multithreaded JDBC Architecture
Time & Space Complexity	O(1) average lookup / O(N + T) storage	O(log M) indexed database access / O(1) auxiliary memory
Data Safety	Temporary memory with periodic file backup	Reliable database storage with transaction support
Concurrency Support	Java synchronization for concurrent booking	Database-level concurrency and transaction management
Duplicate Booking Prevention	synchronized seat allocation and in-memory validation	Database constraints and transactional updates
Deadlock Mitigation	Controlled synchronization and limited critical sections	Database lock management and transaction control
User Interface	Native Java AWT desktop application	JDBC database backend for persistent storage
Recommended Role	Fast processing and local data backup	Permanent storage of passenger, ticket, and reservation records

15. Design Decisions, Project-Specific SDG Relevance & Reflection

- **Design Choice:** The system uses HashMap for fast searching of trains and tickets using unique IDs. Object-Oriented Programming helps organize passengers, trains, coaches, seats, and tickets into separate and reusable classes.
- **Concurrency Design:** Java multithreading and synchronized methods are used to handle simultaneous booking requests. This prevents race conditions and ensures that the same seat cannot be booked by multiple passengers at the same time.
- **SDG 8 (Decent Work & Economic Growth):** The system improves the efficiency of railway reservation operations by reducing manual work and providing faster ticket management.
- **SDG 9 (Industry, Innovation & Infrastructure):** The project demonstrates modern digital transportation infrastructure through Java programming, database connectivity, multithreading, and automated reservation management.